Milestone 2: The Autonomous Shopping Cart

Tyler Potsiadlo, Tianxiang Liu, and Vedarsh Mishra
CSCI3387.01 Topics in Computational Intelligence

March 17, 2026

1 Evolution from Milestone 1

Based on feedback and feasibility analysis, several core concepts from Milestone 1 have been updated:

- **Scope Shift (Stroller to Shopping Cart):** We have pivoted from a baby stroller to a grocery store shopping cart. This provides a highly controlled environment (clean, flat floors, distinct aisles, rigid turning angles) and removes the critical safety risk of transporting an infant. It also serves as a stronger intermediate commercial product, as the shopping mall can scale out the fixed cost of the cart while increasing accessibility for shoppers.
- **Hierarchical Control Architecture:** Instead of forcing a single neural network to handle complex 2D spatial reasoning and turning, we have adopted a regime-switching architecture. We are isolating the Machine Learning task strictly to 1D distance maintenance, while handling turning through a deterministic kinematic S-Curve macro and a Proportional-Derivative micro-correction controller.
- **Simulation Development:** We have successfully built a custom 2D kinematic simulation environment in Python/Gymnasium to train our RL agent safely before transferring to real-world RC hardware.

2 Introduction

The problem we are addressing sits within the broader context of autonomous vehicles and assistive robotics: hands-free cargo transport in pedestrian environments. Specifically, we are designing an autonomous front-leading shopping cart that navigates ahead of a user, allowing them to browse shelves and place items into the cart without manually pushing it.

Following a user from the front—rather than trailing behind—presents unique control and prediction challenges. The cart must continuously estimate the user's intended trajectory using posture cues (e.g., shoulder rotation) and execute non-holonomic maneuvers to change lanes without colliding with shelves. Furthermore, safety and collision avoidance are paramount; the system must gracefully handle dynamic obstacles and user unpredictability.

3 Toy Problem: Mock Examples and Edge Cases

To successfully operate in a grocery store, the cart must fluidly switch between different behavioral regimes.

- **Good Behavior - 1D Steady-State Following:** The cart maintains a strict 0.5m to 0.8m leading distance ahead of the user while traveling down a straight aisle, smoothly matching the user's velocity without oscillating.
- **Good Behavior - Cornering and Lane Changes:** When the user turns their shoulders to enter a new aisle, the cart anticipates the turn, calculates a continuous-curvature Dubins path (Clothoid S-curve), and smoothly merges onto the user's new trajectory without spinning in place.
- **Bad Behavior - Dead Reckoning Drift:** The cart executes a turn but fails to use closed-loop correction, permanently drifting at an incorrect angle and eventually colliding with an aisle shelf.
- **Collision Avoidance (Edge Case):** If a shopper steps between the cart and the user, the cart must immediately trigger an emergency stop (velocity = 0). The cart will not attempt to autonomously navigate around the obstacle; instead, it will remain locked until the user manually resets it, ensuring safety in a crowded store.

4 Related Work

Our architecture builds upon prior research in human-following robotics and motion prediction:

- **Rule-Based Smart Carts:** Sanghavi et al. [1] developed a human-following smart cart utilizing ultrasonic sensors and magnetometers. While effective for basic trailing, their purely reactive rule-based approach struggles with the predictive requirements of a front-leading cart.
- **Predictive Leading Navigation:** Wang [2] demonstrated that predicting human walking motion even a short time into the future significantly improves tracking performance, particularly when the robot leads from the front. We incorporate this by using computer vision to track shoulder vectors for early turn detection.
- **RNNs for Human Trajectory:** Moon & Seo [3] explored using Recurrent Neural Networks to predict human trajectories during haptic collaboration. While we currently use Deep Reinforcement Learning for longitudinal control, their insights into biomechanical sway and gait noise inform our Kalman-filtered observation space.

5 Data Sources

5.1 Description of the Data

- **Video data:** We use a single monocular RGB camera mounted on the cart's rear-facing side to record video of the person being followed. Frames are captured at 30 FPS at 2304×1296 resolution. Data is collected in indoor environments (hallways and open rooms) with varied lighting to ensure the model generalizes across conditions.
- **Ground truth relative position:** We collect the ground truth distance and angle using an ArUco marker and OpenCV's `cv2.aruco` library. The person wears a printed ArUco marker, allowing us to accurately measure the 3D relative position of the marker from the camera in each frame via `estimatePoseSingleMarkers`. From the relative position data we extract the distance, horizontal angle, and instantaneous velocity of the person relative to the cart. These are used to create reward signals (collision penalty, distance-tracking reward) during RL training and to supervise the CV distance head.

- **Motor commands:** We log the commanded throttle and steering values sent to the cart at each control step. These commands are synchronized with the camera frames and ArUco-based ground truth so that each observation can be paired with the exact action taken by the controller. This allows us to train and evaluate the reinforcement learning policy on state-action trajectories and to diagnose failure cases such as overshoot, oscillation, or delayed braking.

5.2 Why the Data is Optimal

Our dataset directly provides the two quantities the control system needs: the person's distance and horizontal angle relative to the camera.

- The ArUco marker ground truth gives sub-centimeter 3D pose accuracy, which is far more precise than consumer depth cameras at the 0.5–3 m operating range of the cart. This lets us train and validate the distance head against a reliable reference.
- YOLOv8 Pose keypoints (shoulders and hips) provide the biomechanical cues—shoulder rotation and shoulder–hip twist—that are mathematically necessary to predict turn intent before the person's trajectory visibly changes direction.
- The horizontal angle of the person is derived geometrically from the bounding-box center and the camera's focal length (obtained during calibration), giving the RL agent a polar-coordinate observation (r, θ) that naturally maps to the cart's steering and throttle actions.
- Collecting our own video in the target environment (indoor, flat floors, consistent lighting) avoids the domain gap that would arise from using a public outdoor pedestrian dataset.

5.3 How the Data Will Be Used

The visual data will be passed through our perception module to extract two continuous variables:

1. The user's distance (Z) for the RL agent.
2. The user's lateral offset (X) and shoulder rotation (θ) for the PD alignment controller.

Additionally, we are actively generating simulated kinematic data via our custom `Vanguard2D` Gymnasium environment, which generates millions of frames of synthetic walking trajectories to train the RL agent.

6 Machine Learning Methods

To solve the problem, we have decoupled the system into a hierarchical control architecture, blending deep reinforcement learning with classic robotics control theory.

6.1 Reinforcement Learning: Proximal Policy Optimization (PPO)

Algorithm Details: We utilize PPO, an actor-critic policy gradient method, to control the cart's acceleration. PPO optimizes a clipped surrogate objective function to prevent destructively large policy updates:

$$L^{CLIP}(\theta) = \hat{E}_t \left[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t) \right] \tag{1}$$

where $r_t(\theta)$ is the probability ratio of the new policy over the old, and $\hat{A}_t$ is the estimated advantage function.
Why Selected: PPO is highly sample-efficient in continuous action spaces (like acceleration) and provides stable convergence compared to standard DDPG or SAC algorithms.
Implementation: PPO receives an observation vector containing distance error and relative velocities. It outputs a 1D continuous action $[-1.0, 1.0]$ mapped to the cart's linear acceleration limits.

6.2 Computer Vision: YOLOv8 Pose Estimation

Algorithm Details: We use a pretrained YOLOv8 Pose Nano model as the frozen backbone of our perception pipeline. This single-stage detector produces each person's bounding box and 17 COCO body keypoints in one forward pass. On top of the frozen backbone, we train lightweight heads to extract the quantities the RL agent needs:

- **Distance** — estimated from bounding-box size features, supervised by ArUco ground truth.
- **Angle** — the horizontal angle of the person relative to the camera, computed geometrically from the bounding-box center and the camera's focal length.
- **Turn intent** (optional) — a classifier over a temporal window of shoulder and hip keypoint angles, predicting left, right, or straight.

The exact architecture of each head (e.g., linear model vs. small MLP) will be determined after initial experiments.
Why Selected: YOLOv8 Pose Nano has only 3.3 M parameters. Its single-pass design avoids the latency and complexity of chaining a separate detector and pose network. We are not sure about the exact latency the model will cause but we have backup plan for the vision model that are smaller and faster but can only handle one person and low resolution video.
Implementation: Each frame is passed through YOLOv8 Pose, and the highest-confidence person is selected as the follow target. The distance and angle outputs form a polar-coordinate observation (Z, α) that is passed to the RL agent. If enabled, the turn-intent signal is forwarded to the deterministic lane-change controller.

6.3 Deterministic Kinematics (Non-ML Module)

To handle turns, we mathematically derive a continuous-curvature lane change (Clothoid sequence) that temporarily overrides the RL agent. This is followed by a Proportional-Derivative controller that minimizes the Cross-Track Error (CTE):

$$\omega_t = -K_p(\theta_{error}) - K_d(CTE) \tag{2}$$

This ensures the cart perfectly aligns with the user's new aisle path before handing control back to the 1D PPO agent.
The reason we implement this override is because a human can turn in place, while a cart, due to its movement path, has a turn radius. During this turn, the cart loses sight of its target person. However, knowing which direction the person has turned (our first implementation will assume only turns within the neighborhood of 90 degrees), the cart can use its velocity to correct its path to place itself back in front of the target on the correct trajectory.

7 Success-Assessment Plan

We will evaluate the system's success across both simulated and real-world metrics:

1. **Longitudinal Error (RL Performance):** Measured by the average variance from the target 0.5m leading gap during steady-state straightaways.
2. **Cross-Track Error (CTE):** Measured by the maximum lateral deviation from the user's exact trajectory during and immediately after a cornering maneuver.
3. **Collision / Kill-Switch Rate:** The frequency at which the cart allows the gap to exceed 3.0m or drop below 0.3m.
4. **Vision Metrics (CV Performance):** The perception pipeline must run at $\geq$10 FPS end-to-end on the target compute board. Distance estimation error must be $\leq$0.1 m (MAE) over the 0.5–3 m operating range, validated against ArUco ground truth.

8 Plan and Strategy

Our task breakdown bridges simulation to in-real-life prototype:

- **Task 1: Simulation & RL Training (Tyler)** - Finalize the `Vanguard2D` Gymnasium environment. Train the PPO agent to converge on the 1D distance-keeping task with noisy simulated inputs.
- **Task 2: Vision & Perception (Jimmy)** - Build and train the computer vision module to extract depth, CTE, and shoulder heading from live camera feeds. Evaluate model latency.
- **Task 3: Hardware Integration (Vedarsh)** - Assemble the RC prototype chassis. Integrate the compute board, depth camera, and motor controllers.
- **Task 4: System Merge & Real-World Testing (All)** - Port the trained PPO weights and kinematic macro logic onto the physical hardware. Test in the 3rd-floor hallways.

9 Contributions

Day 1 to Milestone 1:

- **Tyler:** Researched RL candidates. Drafted initial problem statements and edge cases for the M1 report.
- **Jimmy:** Explored CV tracking frameworks and drafted the CV training strategy section.
- **Vedarsh:** Drafted hardware requirements, researched RC car platforms, and helped structure the evaluation matrix for project selection.

Milestone 1 to Milestone 2:

- **Tyler:** Developed a custom `gym.Env` in Python. Programmed the non-holonomic physics engine, closed-form Dubins path S-curve math, and the PD alignment controller. Ran initial PPO training loops using Stable Baselines3.

- **Jimmy:** Designed the full computer vision pipeline architecture: selected YOLOv8 Pose Nano as the frozen backbone, defined the distance and angle estimation (geometric computation from bounding-box center and camera focal length), and the optional turn-intent classifier.
- **Vedarsh:** Set up the Raspberry Pi camera and headless development workflow, validated live camera capture on the compute board, and began integration planning for the motor-control / servo-control subsystem.

References

[1] S. Sanghavi, P. Rathod, P. Shah, and N. Shekokar, "Design and Implementation of a Human Following Smart Cart," *Proceedings of the Third International Conference on Smart Systems and Inventive Technology (ICSSIT)*, IEEE, 2020.

[2] A. Wang, "Human Motion Prediction Applicable to Human-Following Robot," *Doctoral Thesis*, 2022.

[3] H.-S. Moon and J. Seo, "Prediction of Human Trajectory Following a Haptic Robotic Guide Using Recurrent Neural Networks," *Proceedings of the IEEE World Haptics Conference (WHC)*, pp. 157-162, 2019.

[4] Google, "Gemini," Large language model used for document formatting and structural organization, 2026. [Online]. Available: https://gemini.google.com